WCode: Revision-Bound Evidence for Stateful Software Agents

Francis Du

6 November 2026

Abstract. Software agents operate through sequences of repository reads, edits, checks, and acceptance decisions. Evidence produced at one step can become inapplicable after a code or design change, and a successful check from one producer can obscure a conflicting result from another. We present WCODE, a Rust runtime that makes these boundaries explicit. Its central abstraction is revision-bound evidence: observations retain their subject, producer, authority, coverage, and code–design revision, while policy determines whether they can support a current operation. The runtime combines bounded source delivery, optimistic file mutation, verification reuse, and producer-aware reconciliation. We specify the resulting admission contract and examine its implementation using a frozen executable. A model-free diagnostic covers 60 synthetic scenarios in four languages, three response budgets, two harness-cache phases, and seven repetitions, yielding 2,520 measured queries. Twenty-two selected contract counterexamples replay successfully on that same executable, and eleven operational controls pass. Every diagnostic condition delivers all 686 required source instances and all 406 eligible editing inputs. Nevertheless, serialized decision-record exposure varies from 0% to 70%, and removing writability from a readiness predicate creates seven false positives among 413 answerable observations per condition. The results show why successful retrieval and readiness for action need separate measurements. They support the tested runtime contracts; generalization to real-repository issue resolution remains an empirical question.

1 Introduction

Repository-level software agents increasingly combine code search, editing, command execution, and feedback within one workflow. SWE-bench measures the ability of complete systems to resolve real repository issues, while SWE-agent and OpenHands demonstrate the importance of the interfaces and execution environments surrounding a language model [3, 7, 11]. These systems make a second question consequential: when an agent acts on a previously observed result, what connects that result to the software state being changed?

Consider an agent modifying session-refresh behavior. It locates the correct function and obtains its source. Another worker then updates the same file. A test result remains available from the earlier revision, a second verifier reports a pass for a narrower target, and the implementation task is marked complete. None of these observations is necessarily false. The error arises if their distinct meanings are collapsed into a single indication that the current change is ready. Localization does not establish that an edit's source is current; an executed check does not establish coverage of every requirement; task completion does not establish acceptance.

This paper studies that coordination problem at the runtime boundary. We use *evidence* to mean a typed observation about an artifact or operation, with recorded provenance and conditions of applicability. Evidence may remain useful as history after it ceases to justify a current operation. The key distinction is therefore between retaining an observation and admitting it into a present decision. This distinction matters whenever an agent, a human editor, and background checks share evolving software state.

We present WCODE, an implemented Rust runtime exposed through the Model Context Protocol. It maintains declared software intent, code observations, verification records, and work state. Its organizing rule is that an operation consumes evidence with explicit applicability conditions. Reading returns original source and file identity; mutation checks the current file against the observed identity; verification records the revision and authority under which a check ran; reconciliation preserves relevant disagreements between producers. The same rule also determines what the interface reports as missing or uncertain.

The research question is: *How can a shared software-agent runtime preserve the applicability of evidence across bounded observation, concurrent modification, and verification?* We examine it through three contributions:

1. An explicit admission contract relating evidence to code–design revision, target scope, authority, and policy. The contract distinguishes current support, historical observations, and unresolved producer conflicts.
2. A runtime implementation that carries these distinctions across context packing, guarded single-file edits, verification execution and reuse, and reconciliation. A running example shows why preserving the fields at each boundary is necessary.
3. An auditable diagnostic evaluation separating source delivery, edit-input readiness, operational controls, and descriptive cost. A retained executable and source archive identify the primary run; a predicate-removal analysis exposes a readiness error hidden by saturated retrieval scores.

The contribution is a composition of established mechanisms around a shared operational contract. Hashing, provenance, architecture conformance, and access control each have substantial prior work; Section 7 locates the runtime within those traditions. Our evaluation is

correspondingly specific. It invokes no language model and establishes behavior on developer-authored scenarios rather than improved patch success. This scope allows us to inspect failures at tool boundaries without attributing them to model reasoning, while leaving downstream effectiveness to a separate experiment.

2 An Evidence Admission Contract

2.1 State and observations

Let the workspace state be

$$S = (D, C, P, G, E, W), \tag{1}$$

where D is declared design, C is code, P is applicable policy, G is the observed software graph, E is the evidence history, and W is work state. Its revision is $\rho(S) = (\rho_C, \rho_D)$, with an optional design component. Policy has a separate binding; it is not a third revision field. Keeping design in the revision matters because a changed acceptance criterion can invalidate the use of a previous result even when the code bytes are unchanged.

For exposition, represent an evidence record as

$$e = (u, k, p, \rho, \sigma, a, o, \pi, \tau), \tag{2}$$

where u is the subject, k the evidence kind, p the producer, ρ the captured revision, σ the covered targets, a the authority, o the outcome, π the relevant policy binding, and τ an ordering or execution receipt. This tuple summarizes checks distributed across the implementation; it is not a claim that every subsystem has one identical record schema.

Provenance and authority serve different purposes. A producer identifier records who supplied an observation. Authority determines which claims that entry point is allowed to establish. A self-reported producer name or confidence score cannot turn external evidence into native verification. Similarly, automated policy authorization and an explicit human approval are distinct evidence kinds. Replacing either distinction with a generic "success" would change the meaning of the decision.

2.2 Applicability and producer conflicts

For a request q, define the applicable subset

$$E_q(S) = \{e \in E : \mathrm{App}(e, S, q)\}, \tag{3}$$

$$\begin{aligned}\mathrm{App}(e, S, q) = {} & \mathrm{Complete}(e.\rho) \\ & \wedge (e.\rho = \rho(S)) \\ & \wedge \mathrm{Relevant}(e.\sigma, q) \\ & \wedge \mathrm{Authorized}(e.a, P, q) \\ & \wedge \mathrm{Compatible}(e.\pi, P, q).\end{aligned} \tag{4}$$

The predicates are operation-specific. Relevance admits observations about the request's targets; exact coverage is checked when a record supports an individual obligation. Compatibility concerns the policy conditions relevant to that evidence kind. An incomplete revision fingerprint cannot establish exact-revision native evidence. Retaining such a record for inspection does not make it applicable.

Applicability alone does not resolve conflicting records. We reduce records within keys that retain subject, evidence kind, producer, and applicability scope. A newer record can supersede an older one only under the applicable typed-receipt and coverage rules. Let L_q denote the resulting retained records. A global "latest result wins" reduction is unsuitable: a pass from producer p_2 must not erase an applicable failure from p_1.

Let Q be the nonempty set of required checks, A the additional configured acceptance gates, and $\bar{S} = (D, C, P, G, W)$ the state excluding evidence history. A simplified admission rule is

$$\begin{aligned}\mathrm{Accept}(S, q) \iff {} & Q \neq \varnothing \\ & \wedge\ \forall t \in Q : \mathrm{Pass}(t, L_q) \\ & \wedge\ \nexists e \in L_q : \mathrm{Fail}_q(e) \\ & \wedge\ \mathrm{GatesSatisfied}(A, \bar{S}, L_q).\end{aligned} \tag{5}$$

Here, $\mathrm{Pass}(t, L_q)$ requires applicable support whose coverage, kind, authority, and receipt satisfy check t; an outstanding skipped check cannot supply it. $\mathrm{Fail}_q(e)$ denotes a blocking failure under the request's policy, rather than every negative or advisory observation. Configured gates consult evidence only through L_q. The implementation distinguishes check aggregation from broader reconciliation and acceptance. Equation (5) makes their logical dependency explicit rather than replacing them with a single execution-status flag.

Proposition 1 (Preservation under irrelevant additions)**.** *Fix $\bar{S}$, the request q, the required checks and gates, and the reduction rules. Adding inapplicable evidence does not change admission. If a retained applicable blocking failure from producer p_1 remains, adding a passing record from a different producer p_2 cannot make admission true.*

Proof. Inapplicable records are removed before reduction and therefore leave L_q unchanged. All other arguments to the checks and gates are fixed, so admission is unchanged. Records from distinct producers occupy distinct reduction keys, so the added pass cannot supersede the blocking failure. The failure makes the third conjunct of Equation (5) false. □

The proposition concerns the specified admission rule under truthful, complete, correctly bound inputs. It does not establish that a test is adequate, that a producer is trustworthy, or that an implementation is formally verified. In particular, a captured revision label alone cannot prove that a process read an immutable snapshot throughout execution. These assumptions separate what the contract guarantees from what evidence capture must establish.

2.3 A running change scenario

Table 1 gives a schematic trace for the session-refresh example. The identifiers are illustrative, not an addition-

Table 1. Illustrative evidence transitions for one change.

Event	Consequence for the next operation
Read h_0	The response identifies the function, original body, and current file hash.
Peer edits	File state becomes h_1; an edit still expecting h_0 must refresh its input.
Old pass	Evidence bound to ρ_0 remains historical when the current revision is ρ_1.
p_1 fails	A current failure remains visible for its subject and scope.
p_2 passes	A pass from another producer does not supersede p_1's failure.
Task ends	Completion updates work state; acceptance still requires applicable checks and gates.

al empirical task. The trace exposes two independent validity boundaries: the file observation used by an edit and the revision-bound support used by acceptance.

There is no universal ordering in which a source excerpt, a test pass, and a human approval are interchangeable amounts of confidence. They support different predicates. The runtime consequently exposes both the observations and the remaining obligations, allowing a caller to choose a targeted re-read, re-verification, or review instead of inferring readiness from an undifferentiated score.

3 Runtime Design and Implementation

wcode implements the contract through four connected responsibilities: observing software state, constructing bounded context, admitting mutations, and reconciling verification evidence. Figure 1 shows their relationship. The runtime does not prescribe the caller's model or planning policy; it constrains the meaning of the operations available to that caller.

3.1 Observing design and implementation

Declared design represents requirements, components, constraints, decisions, and acceptance criteria. References connect components to files or symbols, and requirements to implementation and verification targets. These references provide traceability: resolving a symbol establishes a mapping, while behavior still requires appropriate verification.

The software graph joins declarations with files, symbols, tests, risks, and evidence. Observed relationships retain their provider, precision, and revision. Syntax-derived structure and fresh semantic-provider results therefore remain distinguishable. A bounded traversal reports incomplete coverage; ambiguous symbol matches or multiple component owners remain candidates. For architectural comparison, the absence of an observed edge is recorded as unverified actual structure when the observation is insufficient. It is not converted into a definite missing dependency.

Design admission is scoped to initialized product designs. In that setting, invalid design state or missing implementation ownership can block a mutation. The rule does not require every unrelated workspace to define a product model. This scope is important: applying a design gate without its initialization precondition would turn a useful project contract into an unexplained global prohibition.

Algorithm 1 Guarded edit admission (contract abstraction)

Input: path f, observed hash h, requested edit Δ
Output: committed hashes or a typed rejection

1. Check applicable workspace, design, and write policy.
2. Resolve f and acquire its per-file mutation lock.
3. Re-resolve f; validate path and link conditions.
4. Read current bytes x. If $\mathrm{H}(x) \neq h$, reject as stale.
5. Construct $x' = \Delta(x)$; validate edit preconditions and proposed content. Reject on failure.
6. Atomically replace the single file with x'; release the lock; return $\mathrm{H}(x), \mathrm{H}(x')$.

The lock is released on every exit path. Independent files in a batch have independent outcomes.

3.2 Bounded context as an output contract

The context interface assembles task targets, original source, file metadata, graph relationships, available checks, and work state. Selection prioritizes explicit targets and useful executable definitions; independent source items can be loaded concurrently. Packing then compacts reconstructible metadata and trims lower-priority content to satisfy an explicit response budget.

For serialized response y, the accounted size is

$$b(y) = \left\lceil \frac{|\,\mathrm{JSON}(y)|_{\mathrm{bytes}}}{4} \right\rceil . \tag{6}$$

The requested bound is $b(y) \leq B$. This byte-based proxy is an interface accounting rule, not a model tokenizer. A complete source window retains original content and line bounds; summaries do not stand in for missing original source. Readiness is refreshed after packing because a symbol found internally may be absent from the delivered response. Final serialization is checked against the budget, with an error if the bound cannot be met.

This output-centered design creates a testable invariant: readiness must be supported by what the caller actually receives. An internal retrieval hit cannot establish delivered-body coverage, and a current hash cannot compensate for an omitted body. Section 5 evaluates these predicates separately.

3.3 Optimistic mutation at a defined scope

An existing-file mutation supplies an expected SHA-256 hash. Admission checks write policy, resolves the path, acquires its per-file lock, resolves the path again, rejects unsupported link conditions, and compares the current hash with the expected hash. The runtime validates the proposed content before an atomic single-file replacement and returns before/after hashes. Algorithm 1 abstracts this sequence.

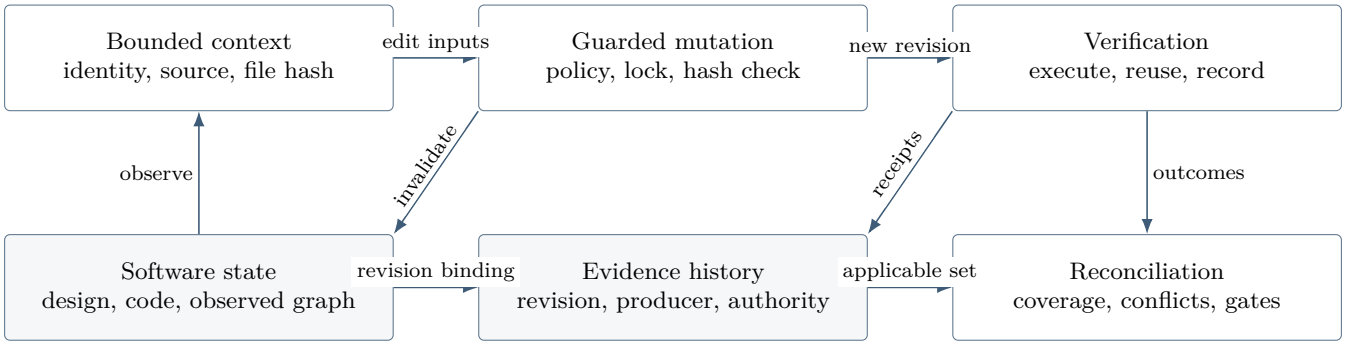


Figure 1. The runtime carries evidence across operation boundaries. Source identity constrains mutation; revision and provenance constrain verification reuse and reconciliation. The caller receives operation results and remaining obligations. Historical records remain inspectable after they lose applicability.

Under cooperating writes protected by this lock, two requests based on the same hash cannot both commit over successive byte-changing states: after the first changes the bytes, the second sees a different hash and fails its precondition. This consequence assumes the first write changes the hash and excludes a hash collision. A no-op write need not invalidate the second request. External editors need not cooperate with the lock, and the batch API is not a multi-file transaction. Callers must inspect partial outcomes and refresh after conflicts.

Command execution has a separate policy boundary. Permission to edit a file does not confer permission to launch an arbitrary process. Keeping file and process admission distinct makes rejection attributable to the relevant operation instead of silently widening authority.

3.4 Verification and reconciliation

Verification captures its input revision before executing a plan. Reports distinguish executed, reused, failed, and skipped checks. The aggregate execution pass requires at least one check, no failures, and no outstanding skipped checks. Eligible in-flight work can be coalesced for a matching revision, while each caller receives an explicit result. Reuse points to existing exact-revision evidence; it does not manufacture a new execution receipt.

Native recording rejects revision fingerprints marked as partial. Trusted entry points assign evidence authority. Producer-aware reduction retains relevant failures until the permitted supersession conditions are met. Consequently, a work item can be complete while its acceptance obligations remain unresolved.

Reconciliation connects declared changes, affected components, implementation tasks, verification plans, and acceptance gates. Its operator-facing state preserves missing mappings, conflicting results, and outstanding review. A design-only change can indicate implementation lag; a mapped code change without a design change can request review without asserting that every implementation-only refactoring is wrong. Runtime queue observations remain advisory when they lack source-revision binding. These distinctions prevent operational activity from being mistaken for accepted software.

4 Evaluation Design

We evaluate the implemented boundaries through four questions: (RQ1) do selected contract tests and operational controls preserve their stated boundaries; (RQ2) does bounded context deliver its annotated source obligations; (RQ3) which readiness errors are hidden by weaker predicates; and (RQ4) how do response size, visible decision records, and latency vary across the tested budgets? The unit under study is a runtime call, without a language-model invocation.

4.1 Corpus and execution protocol

The diagnostic contains 60 synthetic, developer-authored scenarios: 27 Rust and 11 each in Go, TypeScript, and Python. Fixtures range from 26 to 644 files. Each scenario has explicit Gold source obligations, including path-qualified identities and original source ranges. Among the 59 answerable scenarios, the union of required original source is small: 61–220 bytes per scenario, with a median of 162 bytes. One scenario is unanswerable and one is answerable but read-only; 58 are eligible for editing. These properties make the corpus useful for contract diagnostics but easy for source-delivery recall.

We run budgets $B \in \{1000, 2000, 4000\}$, two harness-cache phases, and seven repetitions per scenario and condition. This yields $60 \times 3 \times 2 \times 7 = 2520$ measured queries, plus 180 separate warm-up queries. “Cold” creates a new tool harness; “warm” uses its primed state. Operating-system caches are not cleared. Budget order is fixed, and phase order alternates by scenario. Repetition characterizes execution variability on the same tasks rather than creating new independent tasks.

The primary run uses an archived source tree at commit `584abd88`, package version 0.9.6, and a retained debug executable. It ran on an Apple M5 system with ten physical/logical cores and 16 GiB memory, macOS 27.2 on arm64. The runtime toolchain probe reported rustc 1.99.1; it is not a separate attestation of the compilation toolchain. The harness is configured for four slots. Host load was not controlled. The complete source and executable identities,

protocol, and recovery history are recorded in Appendix A and the accompanying evidence inventory.

4.2 Delivery and readiness measures

For each condition, the 420 measured attempts contain 686 required source instances, 413 answerable observations, and 406 eligible editable observations. We score the following nested delivery predicates against the final response:

Identity. The annotated path-qualified target is present.
Original body. The required source is delivered without truncation; this predicate already checks the source-window hash.
File metadata. The associated `files[]` entry also reports the current SHA-256.
Editable input. The response satisfies the complete editing-input contract, including write eligibility.

The first three source-instance measures use 686 obligations per condition; editable-input success uses 406 eligible observations. Separately, an attempt-level readiness analysis compares positive classifications among the 413 answerable observations. Keeping these denominators explicit prevents source recall from being reported as edit readiness.

We also measure the presence of a serialized `DecisionBatch`, an optional structured decision record in the final response. Its denominator is all 420 attempts. Exposure measures what the caller can inspect; it is not decision correctness. The raw report additionally retains shadow context-sufficiency scoring, which is a separate diagnostic and does not change the exposure measure.

4.3 Controls, ablation, and timing

Eleven operational controls cover four language-specific syntax graphs, guarded editing and invalidation, read-only admission, competing same-hash writes, workspace boundaries, partial-graph reporting, syntax-cache pressure, and workspace-relative verification singleflight. They exercise selected mechanisms alongside the primary trial.

We additionally replay 22 existing selected counterexample tests on the same retained executable, once per exact test name. They cover evidence identity and authority (11), producer conflict (2), dependency ownership (2), scope and observability (2), and scoring (5). Each invocation records its arguments, result, and before/after executable hashes. The source manifests and primary raw report are checked before and after the replay. Earlier executions under another build remain historical; the new replay supplies the results reported here.

The predicate analysis recombines retained per-sample score fields produced by grading the final responses; full serialized tool responses are not archived. It is an observational ablation of readiness definitions, not a runtime experiment that disables a module. For cost, we report serialized bytes and wall-clock latency of the context call. Timing excludes compilation, fixture construction, grading, model inference, network transport, and client retries.

Table 2. Selected counterexamples replayed on the retained executable. Each test is executed once. Counts indicate successful expected behavior, not sampled failure-detection accuracy.

Boundary and tested distinction	Passed
Acceptance: revision, receipt, authority, and approval conditions	11/11
Producer conflict: applicable failures survive another producer's pass	2/2
Dependency ownership: uncertain mappings remain qualified	2/2
Scope and observability: unresolved work and verification state	2/2
Evaluator: grading behavior on constructed counterexamples	5/5
Total	22/22

Table 3 summarizes all 420 observations per condition. Figure 3 instead plots 60 scenario medians, each formed from seven repetitions, so heavily repeated fixtures are not visually presented as independent tasks. We report descriptive distributions without inferential confidence intervals.

5 Results

5.1 Contract and operational boundaries

All 22 selected counterexample tests pass when replayed individually on the same retained executable as the context diagnostic. Each invocation reports one passed test, zero failed tests, and zero ignored tests; none times out. Executable hashes match before and after every invocation, and the 774-path source manifest, all 1003 archived source files, and the primary raw report remain unchanged. Table 2 groups the tested boundaries.

These tests give concrete instances of the contract in Section 2: a result with the wrong revision or authority cannot supply the required support, and a passing record from another producer does not remove an applicable blocking failure. The outcomes validate the selected assertions under the tested fixtures. They do not establish the adequacy of arbitrary verification plans or resistance to adaptive attacks. Earlier executions of these tests used a different build; the replay resolves that provenance difference without pooling the two executions.

All eleven primary controls pass. In the competing-write control, two operations begin with the same observed hash; one commits and one is rejected. The read-only and workspace-boundary controls reject their ineligible operations. Partial-graph reporting retains its incomplete status. Under syntax-cache pressure, the recorded run performs 13,168 trims and still reports 96 files, 96 matches, zero failures, and complete coverage. Workspace-relative verification singleflight passes its targeted duplicate-execution check.

These controls support RQ1 at their stated granularity. For example, the concurrent-write outcome is evidence

for the guarded single-file path under the tested schedule. It does not establish transactional multi-file editing or coordination with every external process.

5.2 Bounded source delivery

For every budget–phase condition, identity, complete original-body delivery, and current file-metadata SHA each satisfy all 686 annotated obligations. All 406 eligible editing attempts satisfy the strict edit-input predicate. Across six conditions, this corresponds to 4116 required source instances and 2436 eligible editing attempts. There are no query errors, response-budget overruns, or warm-up errors.

These saturated outcomes answer RQ2 for the retained corpus. They also reveal the corpus's limited discrimination: its required source is small enough to fit even the lowest budget. A 100% source score therefore says little about how additional budget changes other useful evidence. Figure 2 juxtaposes the saturated delivery measures with decision-record exposure and actual response size to make that distinction visible.

5.3 Nested readiness predicates

For RQ3, the attempt-level identity, body, and body-plus-file-metadata predicates each classify 413 answerable observations as positive per condition. The full editable-input predicate classifies 406. Omitting write eligibility therefore admits seven read-only observations, or $7/413 = 1.69\%$ of answerable observations. Across all six conditions, the 42 affected observations repeat the same read-only fixture; they are not 42 independently discovered vulnerabilities.

The result is small but diagnostic. Correct localization, original source, and fresh hashes can all be present while editing remains inadmissible. Removing the extra file-metadata SHA predicate produces no further separation in this corpus; the preceding body predicate already checks the source-window hash. The analysis therefore does not identify independent causal effects for each nested condition. It establishes a concrete counterexample to equating source availability with edit readiness.

5.4 Budget, visibility, and latency

Serialized `DecisionBatch` exposure is 0/420 at budgets 1000 and 2000, and 294/420 (70%) at budget 4000, in both phases. Thus, equal source recall does not imply equal visibility into the runtime's structured decisions. This finding answers the visibility part of RQ4 without interpreting the exposed records as correct decisions on real tasks.

Across the six conditions, median call latency ranges from 12.948 to 40.186 ms; the largest p95 is 70.913 ms. Warm-phase summaries are lower than their corresponding cold-phase summaries. Larger budgets also have lower measured latency in this run. The latter relationship is not a demonstrated speedup caused by budget: fixed budget order, packing work, shared operating-system caches, and uncontrolled host load are potential confounders. The defensible conclusion is that the call medians and 95th percentiles were in the tens of milliseconds on this host, under the stated debug-build protocol; individual observations and some scenario medians have a longer tail.

The combined result is a three-way separation. Source obligations can be completely delivered, an edit can still be ineligible, and structured decision records can still be absent. Treating any one of these measurements as a universal readiness score would hide an observable difference in the returned interface.

6 Discussion and Validity

6.1 What the contract changes

The central design consequence is that readiness is relational. A record supports a particular operation under a particular state and policy; it does not carry a context-free "valid" bit. Keeping the bindings explicit makes three responses available: accept applicable support, expose an applicable blocker, or identify missing required support. The latter two both prevent admission, but lead to different next actions. A stale source requires another observation; a current failing check requires resolution; a missing approval requires the configured approval process.

This design also separates explanation from authorization. A historical pass can explain why a worker acted, even though it cannot authorize acceptance of a later revision. A syntax edge can guide investigation without establishing semantic conformance. A completed work item can describe progress without satisfying verification. Preserving those records is useful precisely because their role is narrower than acceptance.

The trade-off is additional state and response metadata. Revision bindings, producer keys, and coverage fields consume bytes and require explicit invalidation and reduction rules. The budget experiment shows one manifestation: full source delivery can coexist with missing decision records. Progressive disclosure can expose deeper evidence on demand, but whether it improves human or agent decisions remains to be measured. Interface compactness should therefore be evaluated against the obligations that disappear, not only against response length.

6.2 Threats to validity

Construct validity. Gold source delivery and edit-input readiness are prerequisites for some operations, not measures of patch correctness. The nested predicate analysis uses a reference definition that already includes writability, so its observed error is a diagnostic consequence of omitting that condition. Similarly, serialized decision exposure measures availability for inspection. The retained shadow scores concern the same development fixtures and cannot establish calibrated decisions on independent tasks.

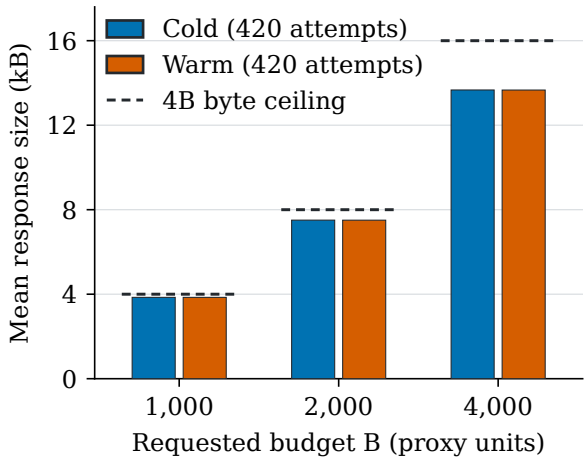
Internal validity. Fixed budget order, debug compilation, and uncontrolled host load limit causal interpreta-

(a) Delivery and decision exposure

Both phases; separate denominators

Measure	1,000	2,000	4,000
Identity recall 686 Gold instances/phase	100%	100%	100%
Complete source 686 Gold instances/phase	100%	100%	100%
Editable inputs 406 eligible attempts/phase	100%	100%	100%
DecisionBatch exposed 420 attempts/phase	**0%**	**0%**	**70%**

(b) Response size and ceiling



Descriptive measurements; 1 kB = 1,000 bytes.

Figure 2. Budget changes different outputs differently. Left: source delivery, editable-input success, and serialized decision-record exposure use distinct denominators (686 source obligations, 406 editable attempts, and 420 total attempts per condition). Rates agree between cold and warm phases; phases are not pooled. Right: mean serialized response bytes remain below the $4B$ ceiling implied by the accounting rule. The lowest budget already saturates the source-delivery measures.

Table 3. Descriptive cost of the frozen primary run. Each row contains 420 measured calls. Latency quantiles summarize calls; the ECDF in Figure 3 uses scenario medians instead. Bytes are serialized JSON bytes, not model tokens.

Budget B	Byte ceiling $4B$	Phase	Mean bytes	p50 (ms)	p95 (ms)	Decision exposure
1000	4000	Cold	3850.14	40.186	70.913	0/420
1000	4000	Warm	3849.71	27.030	51.216	0/420
2000	8000	Cold	7506.04	34.795	61.286	0/420
2000	8000	Warm	7503.97	20.738	34.736	0/420
4000	16000	Cold	13667.22	25.709	56.378	294/420
4000	16000	Warm	13663.93	12.948	25.150	294/420

tions of timing. A fresh harness leaves operating-system caches intact. Before/after source checks and a retained executable improve attribution of the reported run, but do not establish hermetic execution or prove that every process dependency was captured. The 22 counterexample replays share the retained executable but are single, developer-selected tests; they do not sample the space of concurrent schedules or possible failures.

External validity. The corpus is synthetic, authored during development, and organized into related language families. Required source is unusually small, and fixture size does not approximate the dependency and build diversity of independent industrial repositories. Repetitions do not increase the number of independent tasks. There is no fixed-model issue-resolution study, matched retrieval or agent baseline, independent adversarial evaluation, or operator study. Results therefore support the tested contracts rather than general superiority in autonomy, security, or usability.

Implementation boundary. The formalization abstracts implementation intent; no refinement proof connects every code path to the equations. File atomicity is per file, and external editors can escape cooperative locking. Revision equality is only as informative as the captured inputs and fingerprints. These boundaries identify the assumptions under which the contract is useful and the places where stronger isolation or instrumentation would be required.

6.3 A discriminating next experiment

A downstream evaluation should hold the model, task set, budgets, and execution environment fixed while comparing matched interfaces on independent repository issues. It should record patch correctness, stale-edit rejections, evidence-refresh cost, unresolved failures, and false acceptance separately. A mechanism intervention could then disable producer-aware reduction or revision checks under controlled fault injection, measuring whether the hypothesized acceptance errors occur. Such a design would test

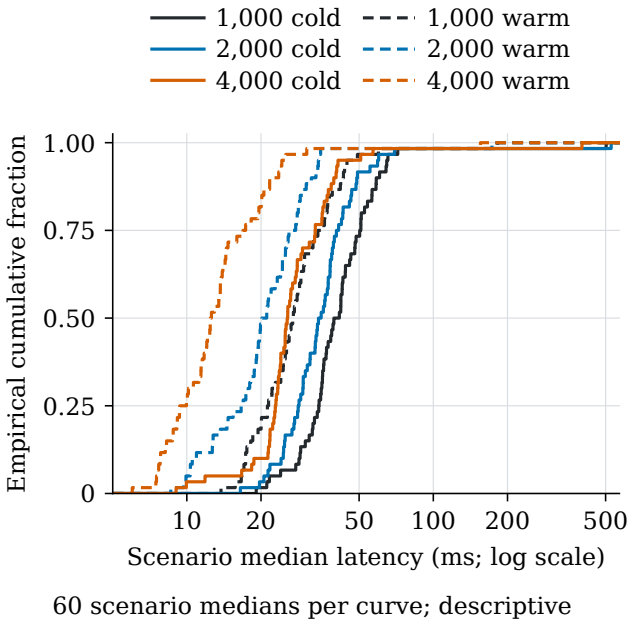


Figure 3. Latency distributions of scenario medians, with a logarithmic latency axis. Each of the six curves contains 60 medians of seven repeated calls. Color denotes budget and line style denotes harness phase. All observed medians, including the upper tail, are retained.

causal benefit; increasing repetitions of the present fixtures would not resolve that question.

7 Related Work

Coding agents and their interfaces. SWE-bench evaluates repository-level issue resolution through real issue–patch pairs and executable tests [3]. SWE-agent demonstrates that the search, read, edit, and execution interface affects an agent's behavior, including through feedback that rejects selected invalid edits [11]. OpenHands provides reusable agent implementations, action–observation events, sandboxed execution, and delegation [7]. Agentless instead organizes localization, repair, and validation into a structured pipeline [10]. These systems motivate studying infrastructure independently of the model's planning policy. WCODE focuses on the applicability of evidence exchanged across operations, while its model-free evaluation isolates runtime behavior from issue-resolution performance.

Architecture conformance and provenance. Software reflexion models connect high-level structure to source-derived models through explicit mappings, exposing agreements and discrepancies [5]. WCODE applies this established distinction within an operational workspace whose declared, syntactic, semantic, and runtime observations retain their different meanings. W3C PROV-DM models entities, activities, responsible agents, and derivation [8]; PROV-AGENT extends provenance capture to agent interactions and workflow tasks, including MCP-based capture [6]. The concern here is when such records can support a current software operation. A record can remain explanatory history while failing revision, scope, or authority conditions. This is a domain-specific admission contract rather than a new provenance standard.

Reuse and incremental computation. Build Systems à la Carte separates scheduling from rebuilding and characterizes reuse through dependency information and verifying traces [4]. The principle that a cached result requires appropriate input conditions is well established. WCODE applies that reasoning to verification and acceptance, preserving the difference between a mapped check, a skipped check, a reused result, and a new execution. Its contribution is their interaction with authority and producer scope in a shared agent-facing runtime, rather than a new incremental-build algorithm.

Repository retrieval. RepoCoder combines repository retrieval with iterative generation, while Repoformer learns when retrieval is likely to improve completion [9, 12]. Both make retrieval quality and cost consequential to code generation. Our delivery contract asks a complementary question: whether a bounded response contains the annotated identity, original source, current file metadata, and eligible editing inputs. These predicates can diagnose the tool layer before a model consumes it. Comparing downstream effectiveness would additionally require matched tasks, budgets, and model configurations.

Execution control and adversarial evaluation. CaMeL separates control flow from untrusted data and uses capabilities to enforce policies on tool-mediated data flows [2]. AgentDojo distinguishes legitimate task utility from attacker success when agents consume untrusted tool outputs [1]. WCODE's workspace checks, command policies, and guarded modifications address specific local admission boundaries. Its selected counterexamples are useful for those boundaries, while adaptive prompt-injection resistance and information-flow security require the broader mechanisms and outcome measures studied in that literature.

Across these lines of research, the building blocks are established. The system studied here makes their shared applicability conditions explicit and carries them across source observation, mutation, verification, and reconciliation. The evaluation measures that composition at concrete interface boundaries.

8 Conclusion

We presented WCODE, a stateful software-agent runtime organized around revision-bound evidence. Its admission contract preserves code–design identity, target coverage, authority, policy, and producer conflicts across operations. The implementation connects bounded source delivery, optimistic mutation, verification reuse, and reconciliation through those distinctions.

The retained executable passes 22 selected contract counterexamples and eleven operational controls. Its diagnostic delivers every annotated source obligation and eligible editing input under all tested budgets. The same data show why delivery alone is insufficient: a read-only observation can satisfy weaker readiness predicates, and structured decision exposure changes even when source recall is saturated. These results establish bounded implementation behavior and an auditable basis for further study. The next empirical step is to test whether preserving these contracts reduces erroneous acceptance and improves real-repository outcomes under matched agent configurations.

References

[1] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-2636. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html.

[2] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. Defeating prompt injections by design, 2025. URL https://arxiv.org/abs/2503.18813v2. arXiv:2503.18813v2.

[3] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[4] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proceedings of the ACM on Programming Languages*, 2(ICFP):79:1–79:29, September 2018. doi: 10.1145/3236774. URL https://simon.peytonjones.org/assets/pdfs/build-systems-original.pdf.

[5] Gail C. Murphy, David Notkin, and Kevin Sullivan. Software reflexion models: Bridging the gap between source and high-level models. In *Proceedings of the Third ACM SIGSOFT Symposium on Foundations of Software Engineering*, pages 18–28. ACM, October 1995. URL https://www.cs.ubc.ca/~murphy/papers/rm/fse95.html.

[6] Renan Souza, Amal Gueroudji, Stephen DeWitt, Daniel Rosendo, Tirthankar Ghosal, Robert Ross, Prasanna Balaprakash, and Rafael Ferreira da Silva. PROV-AGENT: Unified provenance for tracking AI agent interactions in agentic workflows. In *2025 IEEE International Conference on eScience (eScience)*, pages 467–473. IEEE, 2025. doi: 10.1109/eScience65000.2025.00093. URL https://arxiv.org/abs/2508.02866v3.

[7] Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, Xiangru Tang, Mingchen Zhuge, Jiayi Pan, Yueqi Song, Bowen Li, Jaskirat Singh, Hoang Tran, Fuqiang Li, Ren Ma, Mingzhang Zheng, Bill Qian, Daniel Shao, Niklas Muennighoff, Yizhe Zhang, Binyuan Hui, Junyang Lin, Robert Brennan, Hao Peng, Heng Ji, and Graham Neubig. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/a4b6ad6b48850c0c331d1259fc66a69c-Abstract-Conference.html.

[8] World Wide Web Consortium. PROV-DM: The PROV data model. W3C Recommendation REC-prov-dm-20130430, W3C, April 2013. URL https://www.w3.org/TR/2013/REC-prov-dm-20130430/. Edited by Luc Moreau and Paolo Missier; 30 April 2013.

[9] Di Wu, Wasi Uddin Ahmad, Dejiao Zhang, Murali Krishna Ramanathan, and Xiaofei Ma. Repoformer: Selective retrieval for repository-level code completion. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 53270–53290. PMLR, 2024. URL https://proceedings.mlr.press/v235/wu24a.html.

[10] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE), July 2025. doi: 10.1145/3715754. URL https://lingming.cs.illinois.edu/publications/fse2025.pdf.

[11] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37, 2024. doi: 10.52202/079017-1601. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/5a7c947568c1b1328ccc5230172e1e7c-Abstract-Conference.html.

[12] Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. RepoCoder: Repository-level code completion through iterative retrieval and generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2471–2484. Association for Computational Linguistics, December 2023. doi: 10.18653/v1/2023.emnlp-main.151. URL https://aclanthology.org/2023.emnlp-main.151/.

A Artifact Identity and Reproduction

The primary dataset is `frozen-trial/fitness.raw.json`, produced from archived commit `584abd88` (tree `12586d3e`). Table 4 gives short locators; the evidence inventory supplies complete hashes, sizes, and paths. All 1003 archived source files were checked against the export. Before/after manifests contain the same 774 paths and content hashes, and code/design revisions are unchanged.

Table 4. Primary artifact identities (SHA-256 prefixes).

Artifact	SHA-256 prefix
Raw trial report	`cf2575276eef583d`
Retained executable, uncompressed	`2ffaac58a3fe25ef`
Executable gzip archive	`56b39d3c254318ad`
Source tar archive	`1ed5157ce1165927`
Corpus definition	`6cf6d8403c48ac25`
Evaluator definition	`43e5455c5758bd11`
Final run validation	`e4c1887409b26ad6`

The initial frozen invocation exited with code 101 because its report-output directory was missing; it produced no primary raw report. After that process ended, the directory was created and the retained executable was restored, hash-checked, and invoked directly without recompilation. The recovery run exited with code 0 and produced the primary report. The failed invocation remains recorded as a failure. The final validator passes 29 checks and explicitly reconciles two manifest-ordering conventions; the original validator and recovery receipts remain available.

The research-source bundle includes the summary, per-scenario and predicate CSVs, their source-hash manifest, and scripts for regenerating the vector figures. The accompanying evidence archive retains the full raw report, source tar archive, compressed executable, validators, and separate historical runs. Reanalysis uses these retained data without a model service. The separate frozen-contract replay retains one receipt for each of 22 exact tests and verifies the same executable and unchanged source. Historical observations are not pooled into these results.

Re-execution requires a new run record. The preserved executable targets macOS arm64 and contains absolute compile-time paths; rebuilding elsewhere requires a compatible Rust toolchain and dependencies and creates a new executable identity. The artifact documentation provides the command and environment details. A public deposit and an archival identifier should identify an actual released version; neither is implied by the local reproduction package.